FULL-STACK DEVELOPMENT · MULTI-LEVEL

Full-Stack Developer Path

Build the entire application, not just one layer.

Frontend foundations to MERN, MEAN, .NET or Java backend specialization.

16-24 weeks | 4 modules | Professional learning

Explore the learning experience →

Practical learning. Clear progression. Work you can explain.

YOUR NEXT CAPABILITY

What will you be able to do?

Build accessible web interfaces, secure APIs, data-backed applications, automated tests and a production-ready capstone.

- Build responsive web applications
- Develop secure production APIs
- Work with SQL and NoSQL data
- Ship a portfolio-grade capstone

Who this is for

- Students
- Developers switching stacks
- Career returners

Your starting point

- No framework experience required

Your progression at a glance

Learn it. Apply it. Explain it.

Move from guided concepts to practical exercises and a coherent piece of work. Use the course resources to revisit difficult ideas and make your reasoning visible.

THE CURRICULUM

A clear route through the subject.

Every module builds towards practical work. The course outline below is aligned with the academy's current program catalog.

1 Web Foundations

Accessible browser experiences.

- Semantic HTML
- Modern CSS
- JavaScript and TypeScript
- Accessibility and responsive design

2 Frontend Engineering

Component-driven web applications.

- React or Angular foundations
- State and forms
- Data fetching
- Frontend testing

3 Backend Specialization

Choose Node.js, .NET or Java.

- API design
- Authentication and authorization
- Validation and errors
- Backend testing

4 Data and Production

Persist, deploy and operate.

- SQL modeling
- NoSQL patterns
- Docker and CI/CD
- Full-stack capstone

</> Practice with purpose

Build a responsive interface, connect an API and database, implement access rules and prepare a tested deployment.

BUILD SOMETHING THAT DEMONSTRATES YOUR LEARNING

A complete web application

Build a responsive interface, connect an API and database, implement access rules and prepare a tested deployment.

Responsive user interface

API and data model

Tested release and project README

Tools and working methods

HTML, CSS and TypeScript | React or Angular | Node.js, .NET or Java | SQL and MongoDB | Git, testing and cloud

Tools are part of the learning workflow, not partner endorsements. Examples may evolve while preserving the learning outcomes.

Delivery

Hybrid learning · 16-24 weeks

Confirm the current schedule, delivery arrangements and any prerequisites before enrolling.

Completion

A completion certificate is available after the required course work and a passing assessment.

Course completion is not a job, placement or funding guarantee.

Your next steps

1. Review the curriculum and your starting point.
2. Review current enrollment options and delivery details.
3. Start learning, keep your evidence and track your progress.

Open the course experience →

Customer-preview edition. Interactive links open the local academy prototype. Confirm course availability, schedule and commercial terms with the academy.

INSIDE THE LEARNING EXPERIENCE

Keep form validation and server authority separate

A learner submits a project title and repository URL. The interface should give immediate feedback, but the server must remain the authority for stored records.

Understand the concept

Client validation improves the interaction but can be bypassed. Validate again at the API boundary, derive the owner from the authenticated session and reject unauthorized updates. Use parameterized database access. Treat loading, validation failure, duplicate submission and success as explicit interface states rather than assuming every request succeeds.

Worked example

```
POST /projects
Input: { title, repositoryUrl }
Server: verify session -> validate input -> authorize owner
        -> insert record -> return project ID
UI: idle -> submitting -> success | field error | retry
Never accept ownerId from the form as proof of ownership.
```

Interpret the result

The page can show field-specific errors without losing the learner's draft. A successful response identifies the stored project. A network failure should not be displayed as a successful save.

Knowledge check

Can a hidden ownerId field safely establish who owns a new record?

Explanation: No. The server should derive ownership from the authenticated session rather than trusting editable client input.

YOUR PRACTICAL ASSIGNMENT

Build evidence, not just familiarity.

Keep form validation and server authority separate

Your task

- Build an accessible form with retained values after a validation error.
- Implement API validation and an owner-scoped read/update path.
- Test a double click, rejected URL and unauthorized record update.

Submission review criteria

- Server-side validation and ownership enforcement
- Accessible loading, error and success states
- Tests for invalid input and cross-user access

Submit

A working artifact or analysis, a short explanation of your decisions, and evidence that addresses each review criterion.

Reflect

Describe one failed approach, how you diagnosed it, and what you would test next. Identify limitations instead of hiding them.

Your project connection

Build a responsive interface, connect an API and database, implement access rules and prepare a tested deployment.

Use the sample assignment as a stepping stone toward the course project, not as a substitute for the full curriculum.

Explore the worked lesson →

Proposed teaching sample for curriculum and UX review. Full lesson production, instructor review and delivery scheduling remain separate work.